

Database Systems

Spring 2025

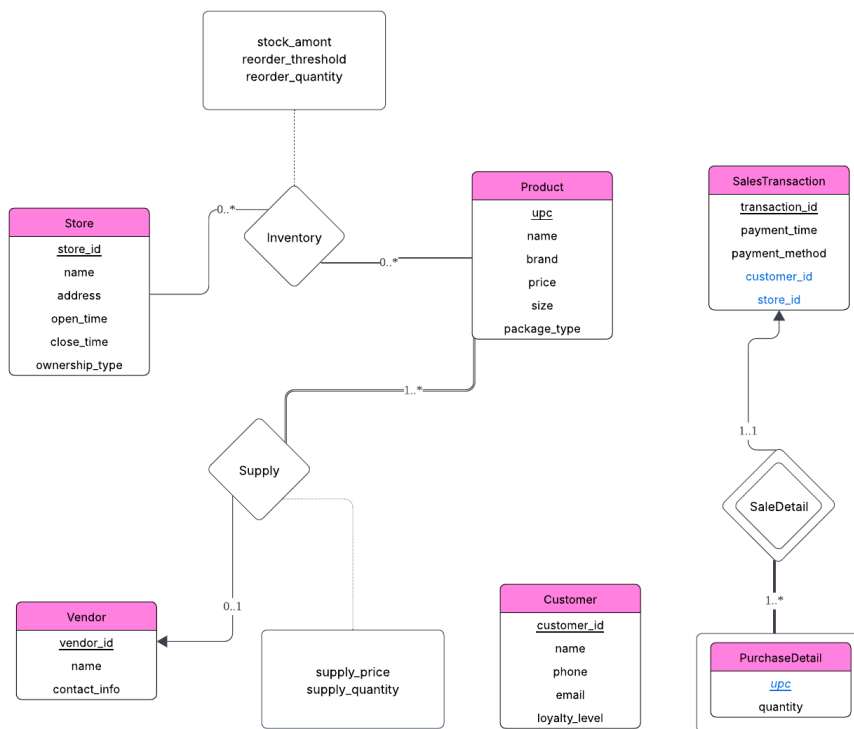
Project 2

20231609 정희선

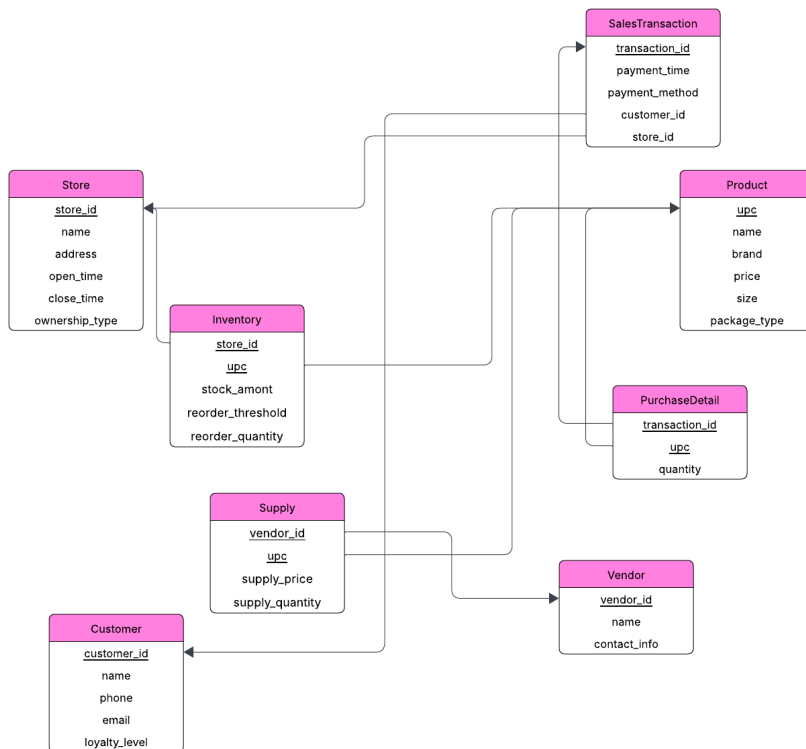
1. Logical Schema Design

• Detailed explanation of ERD to relational schema transformation

우선 project 1에서 설계한 ERD는 다음과 같습니다.



위 ERD 모델을 reduction하여 아래의 relational schema로 만들었습니다.



Reduction에 사용한 규칙은 다음과 같습니다.

- 일반 entity는 relation으로 변환하며, pk는 그대로 유지합니다.
- Weak entity 역시 relation으로 변환하며, 이 relation의 pk는 owner entity의 pk와 discriminator로 구성됩니다.
- 1:1 relationship은 한쪽 entity에 fk를 추가하여 참조합니다. 특히, 양쪽 모두 total participation인 경우 아무쪽이나 fk를 추가할 수 있습니다.
- 1:N relationship의 경우 one participation 쪽의 pk를 N participation 쪽에 fk로 추가합니다.
- M:N relationship의 경우 양쪽 pk를 조합한 새 relation을 생성합니다.
- Weak entity의 relationship은 새 relation을 만들 필요 없습니다.
- 이외에도 다양한 reduction 규칙이 있으나, 위 schema를 구현하는 데 사용한 규칙들만 명시하였습니다.

• Justification for design decisions

primary key 설계 전략

- store_id, customer_id, vendor_id, upc를 pk로 선택하여 비즈니스 데이터 변경에 무관하게 키 값을 유지하도록 하였습니다.

복합 primary key 설계

- Inventory(store_id, upc): 실제 store에서와 같이 "특정 매장의 특정 상품"이라는 개념으로 재고 관리의 기본 단위를 설정하였습니다.
- PurchaseDetail(transaction_id, upc): 하나의 거래에서 동일한 상품을 한 번만 기록하도록 하여 데이터의 중복을 방지하는 pk를 설계하였습니다.
- VendorSupply(vendor_id, upc): 현재 시점의 공급 관계만 관리하여 단순하고, 동일 vendor의 동일 상품 중복 계약을 방지하도록 설계하였습니다.

2. Normalization Analysis

• Functional dependency analysis for each relation

1. Store(store_id, name, address, open_time, close_time, ownership_type)
 - store_id → name, address, open_time, close_time, ownership_type
 - store_id는 pk이므로 BCNF를 만족합니다.
2. Product(upc, name, brand, price, size, package_type)
 - upc → name, brand, price, size, package_type
 - upc는 pk입니다.
 - 이 서비스에서는 모든 브랜드는 여러 상품을 가질 수 있다고 가정합니다. 따라서 brand는 결정자가 될 수 없고, BCNF를 만족합니다.
3. Customer(customer_id, name, phone, email, loyalty_level)
 - customer_id → name, phone, email, loyalty_level
 - phone → customer_id, name, email, loyalty_level
 - customer_id는 Primary Key, phone은 candidate key입니다.
 - 두 FD 모두 결정자가 superkey이므로 BCNF를 만족합니다.
4. Vendor(vendor_id, name, contact_info)
 - vendor_id → name, contact_info
 - vendor_id는 pk이므로 BCNF를 만족합니다.
5. SalesTransaction(transaction_id, payment_time, payment_method, customer_id, store_id)
 - transaction_id → payment_time, payment_method, customer_id, store_id
 - transaction_id는 pk입니다.

- 하나의 고객이 여러 번 결제 가능하므로, 고객 id와 매장 id는 거래의 결정자가 될 수 없습니다.
- 즉, BCNF를 만족합니다.

6. Inventory(store_id, upc, stock_amount, reorder_threshold, reorder_quantity)

- (store_id, upc) → stock_amount, reorder_threshold, reorder_quantity
- (store_id, upc)는 pk입니다.
- 재주문 임계치와 수량은 상품이 같아도 store 별로 관리하므로 다를 수 있다고 가정합니다. 따라서 upc 하나의 속성은 결정자가 될 수 없습니다.
- 즉, BCNF를 만족합니다.

7. Supply(vendor_id, upc, supply_price, supply_quantity)

- (vendor_id, upc) → supply_price, supply_quantity
- 상품마다 고정 가격이라면, 즉 upc → supply_price가 항상 성립한다면 **BCNF를 위반합니다.**
- 이는 대형 유통체인의 경우 본사에서 상품별로 통일된 공급가를 제조사와 계약하여 중앙 집중 구매하는 현실세계의 상황을 반영한 것입니다.

8. PurchaseDetail(transaction_id, upc, quantity)

- (transaction_id, upc) → quantity
- 이 서비스는 한 거래에서 같은 상품이 여러 번 등장하는 것이 아니라, 같은 상품을 한 줄로 묶고 수량을 늘리는 방식이라고 가정합니다.
- 즉, BCNF를 만족합니다.

• **Step-by-step BCNF decomposition process**

BCNF를 위반하는 Supply relation에 대해 decomposition을 진행합니다.

ProductPrice(-- 상품별 고정 가격

upc PRIMARY KEY,

supply_price

)

VendorSupply(— 벤더별 공급 수량만 관리

vendor_id,

upc, -- FK → ProductPrice.upc

supply_quantity,

PRIMARY KEY (vendor_id, upc)

)

Decomposition에 대해서는 아래의 방식을 사용하였습니다.

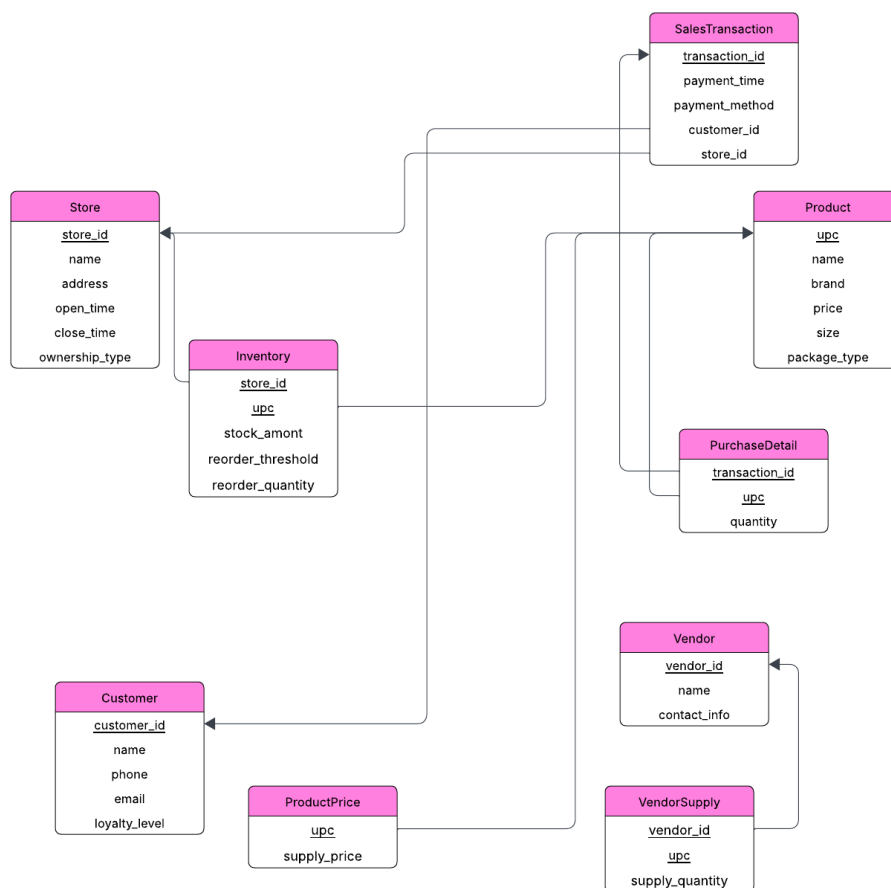
We decompose R into:

- $(\alpha \cup \beta)$
- $(R - (\beta - \alpha))$

$a \rightarrow b$ 가 BCNF를 위반하는 FD인 경우, schema R 을 위와 같은 방식으로 분해합니다.

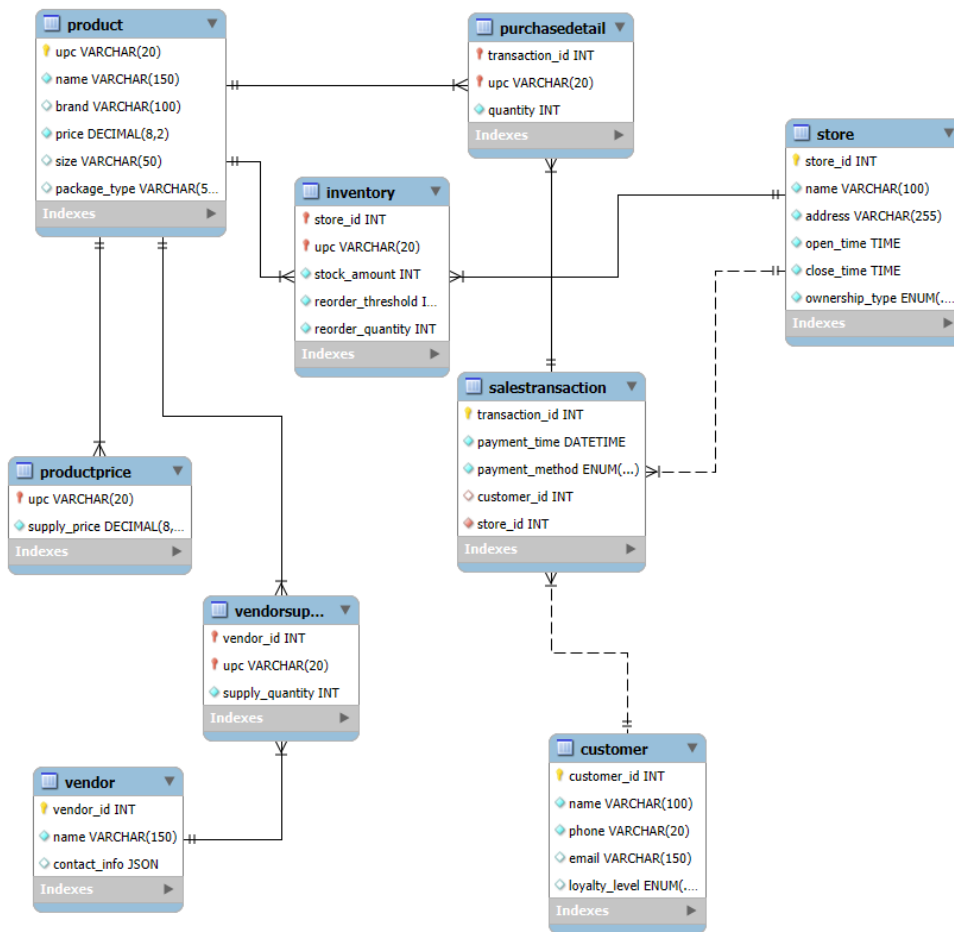
• Proof that final schema satisfies BCNF

BCNF normalization을 마친 최종 logical schema는 다음과 같습니다.



Supply를 제외한 relation에 대해 BCNF를 만족함을 가장 처음에 보였고, 바로 위 과정에서 Supply relation에 대한 분해를 마쳤으므로 모든 relation은 BCNF를 만족합니다.

3. Physical Implementation



이 서비스 DB의 physical schema는 위와 같습니다.

위 schema는 mysql workbench에서 export한 것으로, 부족한 설명은 아래의 추가설명과 각 항목으로 보충하도록 하겠습니다.

- Key 아이콘: primary key를 의미함
- 붉은 색 아이콘: 외래키임을 의미함
- 푸른색 아이콘: not null 제약조건을 의미함

• Data type selection rationale

1. 숫자형 데이터

- store_id, customer_id, vendor_id, transaction_id에서 INT AUTO_INCREMENT: pk용으로, 자동 증가로 유일성을 보장하였습니다.
- DECIMAL(8,2): 정확한 소수점 계산이 필요한 가격 정보에 사용하였습니다.
- INT: 재고량, 수량 등 정수값에 사용하였습니다.

2. 문자형 데이터

- VARCHAR(20): UPC 코드로 고정 길이에 가까우나 유연성 고려하였습니다.
- VARCHAR(100-255): 이름, 주소 등 가변 길이 텍스트에 사용하였습니다.

3. 시간 데이터

- payment_time에서 DATETIME: 정확한 거래 시간을 기록하기 위해 사용하였습니다.
- open_time, close_time에서 TIME: 날짜가 불필요한 매장 운영시간에 사용하였습니다.

4. 특수 데이터

- contact_info에서 JSON: 벤더 연락처와 같이 구조화되지 않은 데이터의 유연한 저장에 사용하였습니다.
- payment_method, ownership_type, loyalty_level에서 ENUM: 제한된 선택지에 사용하였습니다.

• Constraint implementation and business rule enforcement

1. 기본키 제약조건

- 위 physical schema에서 key 아이콘으로 표현되어 있습니다.
- 또한, primary key 설계 전략에 대해서는 위의 Justification for design decision에서 충분히 설명하였습니다.
- AUTO_INCREMENT를 적용하여 고유 식별자를 자동 생성함으로써, 중복 없는 데이터 입력을 보장합니다.
- 실제 비즈니스 시나리오에서, store_id의 경우 매장 이름이 변경되더라도 매장 id는 변하지 않으므로 모든 참조 테이블의 데이터 일관성 유지가 가능합니다.

2. 외래키 제약조건

FOREIGN KEY (store_id) REFERENCES Store(store_id)
ON DELETE CASCADE ON UPDATE CASCADE,
FOREIGN KEY (upc) REFERENCES Product(upc)
ON DELETE CASCADE ON UPDATE CASCADE

- 모든 관계형 테이블(예: Inventory, SalesTransaction, PurchaseDetail)은 외래키를 통해 참조 무결성을 유지합니다.
- 특히 ON DELETE CASCADE 설정을 통해 부모 레코드 삭제 시 관련된 자식 레코드를 자동으로 삭제하도록 하여 데이터 일관성을 자동으로 유지하도록 설계했습니다.
- 이는 실제 비즈니스 시나리오에서 매장 폐점 시 관련 재고/거래 데이터를 자동으로 삭제하여 운영의 편리함을 제공합니다.

3. Check 제약조건

- CHECK (price > 0) -- 가격은 양수
- CHECK (stock_amount >= 0) -- 재고는 음수 불가
- CHECK (quantity > 0) -- 구매 수량은 양수
- 비즈니스 규칙에 따라 특정 속성의 값 범위를 제한하여 데이터의 타당성을 보장합니다.

4. Unique 제약조건

- phone VARCHAR(20) NOT NULL UNIQUE
- email VARCHAR(150) UNIQUE
- 고객 정보에서 phone과 email 컬럼에 대해 UNIQUE 제약조건을 설정하여, 동일한 연락처를 가진 고객이 중복 등록되는 것을 방지합니다.

5. 정규식 제약조건

- CONSTRAINT chk_phone_format CHECK (phone REGEXP '^[0-9-+()#*]{10,15}\$')
- CONSTRAINT chk_email_format CHECK (email REGEXP '^([A-Za-z0-9-_%+]{1,64})+@[A-Za-z0-9-]{1,64}(\.[A-Za-z]{2,4}){1,6}\$')
- 단순한 형식 오류를 사전에 차단하여, 데이터 품질을 높은 수준으로 유지할 수 있게 해줍니다.

6. 기본값 제약조건

- loyalty_level ENUM(...) DEFAULT 'bronze'
- 사용자가 값을 입력하지 않아도 비즈니스 정책에 따라 자동으로 초기 상태를 설정합니다.

• sample data description

샘플 데이터는 실제 store 운영을 반영하며, 다양한 시나리오를 테스트 가능하고, 일관된 비즈니스 규칙을 준수하도록 설계하였습니다.

주요 특징은 아래와 같습니다.

- franchise와 corporate를 혼합한 10개의 매장이 존재함
- 12명의 다양한 등급 고객이 존재함
- 15개의 상품이 존재함
- 거래 내역은 최근 한달 및 현재 분기의 데이터를 포함함 (TYPE 2,3 쿼리 테스트 시나리오)

지원)

- 재고 데이터는 일부 매장에서 재주문이 필요한 상황이 존재함 (TYPE 5 쿼리 테스트 시나리오 지원)

4. Application Development

• Database connectivity implementation details

아래와 같은 코드로 vs code에서 MySQL C Connector를 사용하였습니다.

```
#include <mysql.h>
```

이와 관련된 더 자세한 내용은 README 파일에 작성하였습니다.

• Query implementation

아래 함수를 main에서 호출하여, switch 문으로 사용자의 입력에 따라 다른 type의 쿼리를 처리할 수 있도록 구현하였습니다. 또한 SQL injection을 방지하기 위해 사용자의 입력이 필요한 쿼리는 parameterized query를 사용하였습니다. 각 쿼리를 처리하는 함수는 아래에서 하나씩 설명하도록 하겠습니다.

```

void QueryChoice(MYSQL* conn){
    int choice;
    while(true){
        std::cout << "----- SELECT QUERY TYPES -----\\n\\n";
        std::cout << "      1. TYPE 1\\n";
        std::cout << "      2. TYPE 2\\n";
        std::cout << "      3. TYPE 3\\n";
        std::cout << "      4. TYPE 4\\n";
        std::cout << "      5. TYPE 5\\n";
        std::cout << "      6. TYPE 6\\n";
        std::cout << "      7. TYPE 7\\n";
        std::cout << "      0. QUIT\\n\\n";

        std::cout << "Select: ";
        std::cin >> choice;

        switch (choice){
            case 1:
                handleType1Q(conn);
                std::cout << "\\n";
                break;
            case 2:
                handleType2Q(conn);
                std::cout << "\\n";
                break;
            case 3:
                handleType3Q(conn);
                std::cout << "\\n";
                break;
            case 4:
                handleType4Q(conn);
                std::cout << "\\n";
                break;
            case 5:
                handleType5Q(conn);
                std::cout << "\\n";
                break;
            case 6:
                handleType6Q(conn);
                std::cout << "\\n";
                break;
            case 7:
                handleType7Q(conn);
                std::cout << "\\n";
                break;
            case 0:
                // 프로그램 종료
                std::cout << "Program terminated.";
                return;
            default:
                std::cout << "Invalid choice.";
        }
    }
}

```

- TYPE 1: 상품별 매장 재고 조회

```
void handleType1Q(MYSQL* conn){
    // 사용자 입력에 따라 정보 + 재고 출력하는 쿼리
    std::cout << "----- TYPE 1 -----\n";
    std::cout << "*** Which stores currently carry a certain product (by UPC, name, or brand), and how much inventory do they have? **\n";

    std::string input;
    std::cout << "Enter product identifier (UPC, name, or brand): ";
    std::cin.ignore();
    std::getline(std::cin, input); // 띄어쓰기 포함된 입력처리

    // 1. Statement 초기화
    MYSQL_STMT* stmt = mysql_stmt_init(conn);
    std::string query =
        "SELECT s.store_id, s.name AS store_name, p.upc, p.name AS product_name, i.stock_amount "
        "FROM Inventory i "
        "JOIN Store s ON i.store_id = s.store_id "
        "JOIN Product p ON i.upc = p.upc "
        "WHERE p.upc = ? OR p.name = ? OR p.brand = ?";

    if (mysql_stmt_prepare(stmt, query.c_str(), query.length())) {
        std::cerr << "Prepare failed: " << mysql_stmt_error(stmt) << "\n";
        mysql_stmt_close(stmt);
        return;
    }

    // 2. 파라미터 바인딩
    MYSQL_BIND bind[3]{};
    unsigned long str_len = input.length();

    for (int i = 0; i < 3; ++i) {
        bind[i].buffer_type = MYSQL_TYPE_STRING;
        bind[i].buffer = (void*)input.c_str();
        bind[i].buffer_length = str_len;
        bind[i].length = &str_len;
    }

    if (mysql_stmt_bind_param(stmt, bind)) {
        std::cerr << "Bind failed: " << mysql_stmt_error(stmt) << "\n";
        mysql_stmt_close(stmt);
        return;
    }

    // 3. 실행
    if (mysql_stmt_execute(stmt)) {
        std::cerr << "Execute failed: " << mysql_stmt_error(stmt) << "\n";
        mysql_stmt_close(stmt);
        return;
    }

    // 4. 결과 바인딩
    MYSQL_BIND result[5]{};
    int store_id;
    char store_name[100], upc[30], product_name[100];
    int stock_amount;
    unsigned long name_len[5];

    result[0].buffer_type = MYSQL_TYPE_LONG;
    result[0].buffer = &store_id;

    result[1].buffer_type = MYSQL_TYPE_STRING;
    result[1].buffer = store_name;
    result[1].buffer_length = sizeof(store_name);
    result[1].length = &name_len[1];

    result[2].buffer_type = MYSQL_TYPE_STRING;
    result[2].buffer = upc;
    result[2].buffer_length = sizeof(upc);
    result[2].length = &name_len[2];

    result[3].buffer_type = MYSQL_TYPE_STRING;
    result[3].buffer = product_name;
    result[3].buffer_length = sizeof(product_name);
    result[3].length = &name_len[3];

    result[4].buffer_type = MYSQL_TYPE_LONG;
    result[4].buffer = &stock_amount;

    if (mysql_stmt_bind_result(stmt, result)) {
        std::cerr << "Bind result failed: " << mysql_stmt_error(stmt) << "\n";
        mysql_stmt_close(stmt);
        return;
    }
}
```

```

// 5. 결과 출력
std::cout << "\n--- Query Result ---\n";
std::cout << std::left << std::setw(12) << "store_id"
            << std::setw(20) << "store_name"
            << std::setw(20) << "upc"
            << std::setw(25) << "product_name"
            << std::setw(10) << "stock" << "\n";

while (mysql_stmt_fetch(stmt) == 0) {
    store_name[name_len[1]] = '\0';
    upc[name_len[2]] = '\0';
    product_name[name_len[3]] = '\0';

    std::cout << std::left << std::setw(12) << store_id
              << std::setw(20) << store_name
              << std::setw(20) << upc
              << std::setw(25) << product_name
              << std::setw(10) << stock_amount << "\n";
}

mysql_stmt_close(stmt);
}

```

FROM Inventory i: 재고 테이블을 기준으로 시작

JOIN Store s: store_id 외래키로 매장 정보 조인

JOIN Product p: upc 외래키로 상품 정보 조인

WHERE 절: OR 조건으로 UPC, 상품명, 브랜드 중 하나라도 일치하는 레코드 필터링

1번 쿼리의 경우, 사용자가 악의적 입력을 넣으면 전체 테이블이 노출되는 SQL

injection이 가능하므로 다른 쿼리들과는 다르게 mysql_stmt_prepare 방식을 사용하였습니다.

위 방식에서는 쿼리 템플릿을 통해 입력값을 안전하게 바인딩합니다.

- TYPE 2: 매장별 최고 판매 상품

```

void handleType2Q(MYSQL* conn){
    // 각 store에서 지난 한 달 동안 판매량이 가장 많은 상품 출력
    std::cout << "----- TYPE 2 ----- \n";
    std::cout << "*** Which products have the highest sales volume in each store over the past month? **\n";

    std::string query =
        "SELECT t.store_id, s.name AS store_name, p.upc, p.name AS product_name, SUM(d.quantity) AS total_quantity "
        "FROM SalesTransaction t "
        "JOIN PurchaseDetail d ON t.transaction_id = d.transaction_id "
        "JOIN Product p ON d.upc = p.upc "
        "JOIN Store s ON t.store_id = s.store_id "
        "WHERE t.payment_time >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH) "
        "GROUP BY t.store_id, p.upc "
        "HAVING total_quantity = ( "
        "    SELECT MAX(sub.total_quantity) "
        "    FROM ( "
        "        SELECT SUM(d2.quantity) AS total_quantity "
        "        FROM SalesTransaction t2 "
        "        JOIN PurchaseDetail d2 ON t2.transaction_id = d2.transaction_id "
        "        WHERE t2.store_id = t.store_id "
        "        AND t2.payment_time >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH) "
        "        GROUP BY d2.upc "
        "    ) AS sub "
        ") "
        "ORDER BY t.store_id;";
}

```

```

    if (mysql_query(conn, query.c_str())) {
        std::cerr << "Query failed: " << mysql_error(conn) << "\n";
        return;
    }

    MYSQL_RES* res = mysql_store_result(conn);
    if (!res) {
        std::cerr << "Result retrieval failed: " << mysql_error(conn) << "\n";
        return;
    }

    int num_fields = mysql_num_fields(res);
    MYSQL_ROW row;
    MYSQL_FIELD* fields = mysql_fetch_fields(res);

    std::cout << "\n--- Query Result ---\n";
    for (int i = 0; i < num_fields; ++i) {
        std::cout << std::left << std::setw(20) << fields[i].name; // 열 너비 고정
    }
    std::cout << "\n";

    while ((row = mysql_fetch_row(res))) {
        for (int i = 0; i < num_fields; ++i) {
            std::cout << std::left << std::setw(20) << (row[i] ? row[i] : "NULL");
        }
        std::cout << "\n";
    }

    mysql_free_result(res);
}

```

날짜 필터링: payment_time 인덱스로 최근 한달 거래 필터링
 조인 연산: transaction_id, upc, store_id 외래키 인덱스 활용
 GROUP BY: 매장별, 상품별 판매량 집계
 상관 서브쿼리: 각 매장별로 최대 판매량 계산
 HAVING 절: 최대 판매량과 일치하는 상품만 선택

쿼리를 실행하고 쿼리 결과를 출력하는 부분은 이후 모든 쿼리에서 동일하므로 이후 설명에서는 생략하도록 하겠습니다.

- TYPE 3: 분기별 최고 매출 매장

```

void handleType3Q(MYSQL* conn){
    // revenue = sum(quantity * price)
    std::cout << "----- TYPE 3 -----\n";
    std::cout << "*** Which store has generated the highest overall revenue this quarter? ***\n";

    std::string query =
        "SELECT t.store_id, s.name AS store_name, SUM(d.quantity * p.price) AS revenue "
        "FROM SalesTransaction t "
        "JOIN PurchaseDetail d ON t.transaction_id = d.transaction_id "
        "JOIN Product p ON d.upc = p.upc "
        "JOIN Store s ON t.store_id = s.store_id "
        "WHERE QUARTER(t.payment_time) = QUARTER(CURDATE()) AND YEAR(t.payment_time) = YEAR(CURDATE()) "
        "GROUP BY t.store_id "
        "ORDER BY revenue DESC "
        "LIMIT 1;";
}

```

분기 필터링: QUARTER(), YEAR() 함수로 현재 분기 거래 선택
 매출 계산: quantity * price로 각 거래의 매출 계산
 SUM 집계: 매장별 총 매출 계산
 정렬 및 제한: 매출 기준 내림차순 정렬 후 상위 1개만 선택

- TYPE 4: 최다 공급 벤더

```
void handleType4Q(MYSQL* conn){
    // 가장 많은 상품을 공급한 vendor
    // 그 vendor가 공급한 상품들이 얼마나 판매되었는지
    std::cout << "----- TYPE 4 -----\\n";
    std::cout << "*** Which vendor supplies the most products across the chain, and how many total units have been sold? **\\n";

    std::string query =
        "SELECT v.vendor_id, v.name, COUNT(DISTINCT vs.upc) AS num_products, "
        "SUM(pd.quantity) AS total_units "
        "FROM Vendor v "
        "JOIN VendorSupply vs ON v.vendor_id = vs.vendor_id "
        "LEFT JOIN PurchaseDetail pd ON vs.upc = pd.upc "
        "GROUP BY v.vendor_id "
        "ORDER BY num_products DESC "
        "LIMIT 1; ";
}
```

FROM Vendor v: 모든 벤더를 기준으로 시작

INNER JOIN VendorSupply: vendor_id로 공급 관계 조인

LEFT JOIN PurchaseDetail: upc로 판매 데이터 조인

COUNT(DISTINCT vs.upc): 각 벤더가 공급하는 고유 상품 수 계산

SUM(pd.quantity): 해당 벤더 상품의 총 판매량

GROUP BY v.vendor_id: 벤더별로 집계

ORDER BY num_products DESC: 공급 상품 수 기준 내림차순 정렬

LIMIT 1: 최다 공급 벤더만 반환

- TYPE 5: 재주문 필요 상품

```
void handleType5Q(MYSQL* conn){
    // 각 store 별로 재고가 재주문 임계값보다 낮은 상품을 찾기
    std::cout << "----- TYPE 5 -----\\n";
    std::cout << "*** Which products in each store are below the reorder threshold and need restocking? **\\n";

    std::string query =
        "SELECT s.store_id, s.name AS store_name, "
        "p.upc, p.name AS product_name, "
        "i.stock_amount, i.reorder_threshold "
        "FROM Inventory i "
        "JOIN Store s ON i.store_id = s.store_id "
        "JOIN Product p ON i.upc = p.upc "
        "WHERE i.stock_amount < i.reorder_threshold "
        "ORDER BY s.store_id;";
}
```

WHERE 필터링: stock_amount < reorder_threshold 조건으로 재주문 필요 상품 선별

조인 연산: 매장명과 상품명 정보 조회

인덱스 활용: Inventory 테이블의 복합 인덱스 활용 가능

- TYPE 6: 커피와 함께 구매하는 상품

```
void handleType6Q(MYSQL* conn){
    std::cout << "----- TYPE 6 -----\n";
    std::cout << "** List the top 3 items that loyalty program customers typically purchase with coffee. **\n";

    std::string input;
    // db에 존재하는 coffee product는 black coffee와 milk coffee만 존재한다고 가정
    // loyalty program customers는 loyalty_level != 'bronze'라고 가정
    std::cout << "Enter a product name (black coffee or milk coffee): ";
    std::cin.ignore();
    std::getline(std::cin, input); // 띄어쓰기 포함된 입력처리

    std::string query =
        "SELECT p2.name AS product_name, COUNT(*) AS co_purchase_count "
        "FROM PurchaseDetail pd1 "
        "JOIN SalesTransaction t ON pd1.transaction_id = t.transaction_id "
        "JOIN Customer c ON t.customer_id = c.customer_id "
        "JOIN Product p1 ON pd1.upc = p1.upc "
        "JOIN PurchaseDetail pd2 ON pd1.transaction_id = pd2.transaction_id "
        "JOIN Product p2 ON pd2.upc = p2.upc "
        "WHERE p1.name = '" + input + "' "
        "AND p2.name != '" + input + "' "
        "AND c.loyalty_level != 'bronze' "
        "GROUP BY p2.name "
        "ORDER BY co_purchase_count DESC "
        "LIMIT 3;";
```

Coffee category가 없고, DB에 존재하는 coffee product는 black coffee, milk coffee 두 종류만 있으므로 두 개의 product 중 입력을 받은 결과를 구현했습니다.

또한, loyalty program customers는 존재하는 loyalty_level 중 'bronze'가 아닌 모든 customer라고 가정하였습니다.

셀프 조인: 동일 거래에서 다른 상품 찾기 위한 PurchaseDetail 셀프 조인

고객 필터링: loyalty_level 조건으로 충성 고객 선별

상품 제외: 기준 상품과 다른 상품만 선택

빈도 계산: 함께 구매된 횟수 계산

```
// SQL injection 방지
if (input != "black coffee" && input != "milk coffee") {
    std::cout << "You can only use 'black coffee' or 'milk coffee' as a coffee product." << "\n";
    return;
}
```

또한, 6번 쿼리도 사용자의 입력을 받아 쿼리가 완성되므로 SQL injection의 가능성을 인식하였습니다. 그 결과 위 코드를 입력받은 이후 추가하였습니다.

6번 쿼리에서는 1번 쿼리와 같이 따로 parameterized query를 사용하지 않고 예외 처리를 통해 SQL injection을 방지하였습니다.

그 이유는 이 쿼리의 경우 앞선 가정으로 인해 사용자 검색하는 coffee product가 'black coffee'와 'milk coffee'만 존재하기 때문입니다.

1번 쿼리는 사용자의 입력 경우가 아주 다양하므로 parameterized query 를 사용하였고, 이 경우는 가능한 사용자의 입력 경우가 두가지 뿐이므로 위와 같은 방식을 사용하였습니다.

- TYPE 7: 매장 유형별 상품 다양성

```
void handleType7Q(MYSQL* conn){
    // franchise store 중 가장 많은 종류 보유한 점포 찾기
    // 이후 corporate store 전체의 평균 상품 종류 수와 비교
    std::cout << "----- TYPE 7 -----\n";
    std::cout << "*** Among franchise-owned stores, which one offers the widest variety of products, and how does that compare to corporate-owned st

    std::string query =
        "SELECT "
        "    fs.store_id AS f_store_id, "
        "    fs.name AS f_store_name, "
        "    fs_variety.product_count AS f_product_variety, "
        "    cs.store_id AS c_store_id, "
        "    cs.name AS c_store_name, "
        "    cs_variety.product_count AS c_product_variety "
        "FROM ( "
        "    SELECT i.store_id, COUNT(DISTINCT i.upc) AS product_count "
        "    FROM Inventory i "
        "    JOIN Store s ON i.store_id = s.store_id "
        "    WHERE s.ownership_type = 'franchise' "
        "    GROUP BY i.store_id "
        "    ORDER BY product_count DESC "
        "    LIMIT 1 "
        ") AS fs_variety "
        "JOIN Store fs ON fs.store_id = fs_variety.store_id "
        "JOIN ( "
        "    SELECT i.store_id, COUNT(DISTINCT i.upc) AS product_count "
        "    FROM Inventory i "
        "    JOIN Store s ON i.store_id = s.store_id "
        "    WHERE s.ownership_type = 'corporate' "
        "    GROUP BY i.store_id "
        "    ORDER BY product_count DESC "
        "    LIMIT 1 "
        ") AS cs_variety "
        "JOIN Store cs ON cs.store_id = cs_variety.store_id;";
```

서브쿼리 1: 가맹점 중 최다 상품 보유 매장 선별

서브쿼리 2: 직영점 중 최다 상품 보유 매장 선별

카티션 조인: 두 결과를 조인하여 비교 결과 생성

COUNT(DISTINCT): 중복 제거하여 정확한 상품 종류 수 계산

• Error handling and user interface design

다음의 항목들에 대해 오류를 처리하였습니다.

- 데이터베이스 연결 오류 처리

```
// MySQL 서버 연결
if (mysql_real_connect(conn, server, user, password, database, 0, nullptr, 0) == nullptr) {
    std::cerr << "mysql_real_connect() failed\n";
    mysql_close(conn);
    return 1;
}
```

- 쿼리 실행 오류 처리

```
if (mysql_query(conn, query.c_str())) {
    std::cerr << "Query failed: " << mysql_error(conn) << "\n";
    return;
}

MYSQL_RES* res = mysql_store_result(conn);
if (!res) {
    std::cerr << "Result retrieval failed: " << mysql_error(conn) << "\n";
    return;
}
```


사용자 인터페이스 디자인은 다음과 같습니다.

```
(base) PS C:\Users\lissa\Desktop\2025\3-1\DB\HW2> .\main.exe
----- SELECT QUERY TYPES -----

1. TYPE 1
2. TYPE 2
3. TYPE 3
4. TYPE 4
5. TYPE 5
6. TYPE 6
7. TYPE 7
0. QUIT

Select: █
```

5. Testing and Validation

• Test case descriptions and results

첫 번째 쿼리부터 모든 케이스에 대해 순서대로 출력된 결과입니다.

1.

```
Select: 1
----- TYPE 1 -----
** Which stores currently carry a certain product (by UPC, name, or brand), and how much inventory do they have? **
Enter product identifier (UPC, name, or brand): cola

--- Query Result ---
store_id      store_name      upc              product_name      stock_amount
1             Downtown Branch 1001234567892    cola              15
2             University Store 1001234567892    cola              28
3             City Center     1001234567892    cola              42
4             Metro Station   1001234567892    cola              16
5             Shopping Mall   1001234567892    cola              25
6             Airport Terminal 1001234567892    cola              12
7             Hospital Branch 1001234567892    cola              22
8             Beach Resort    1001234567892    cola              18
9             Mountain Lodge  1001234567892    cola              14
10            Central Plaza   1001234567892    cola              20
```

```
Select: 1
----- TYPE 1 -----
** Which stores currently carry a certain product (by UPC, name, or brand), and how much inventory do they have? **
Enter product identifier (UPC, name, or brand): 1001234567804

--- Query Result ---
store_id      store_name      upc              product_name      stock_amount
4             Metro Station   1001234567804    sandwich          15
5             Shopping Mall   1001234567804    sandwich          28
```

```
Select: 1
----- TYPE 1 -----
** Which stores currently carry a certain product (by UPC, name, or brand), and how much inventory do they have? **
Enter product identifier (UPC, name, or brand): PowerUp

--- Query Result ---
store_id      store_name      upc              product_name      stock_amount
2             University Store 1001234567800    energy drink      15
5             Shopping Mall   1001234567800    energy drink      30
```

2.

```
Select: 2
----- TYPE 2 -----
** Which products have the highest sales volume in each store over the past month? **

--- Query Result ---
store_id      store_name      upc              product_name      total_quantity
1             Downtown Branch  1001234567890    black coffee      14
2             University Store 1001234567890    black coffee      4
2             University Store 1001234567891    milk coffee       4
3             City Center      1001234567890    black coffee      6
4             Metro Station    1001234567890    black coffee      6
5             Shopping Mall    1001234567890    black coffee      12
```

3.

```
Select: 3
----- TYPE 3 -----
** Which store has generated the highest overall revenue this quarter? **

--- Query Result ---
store_id      store_name      revenue
5             Shopping Mall   91800.00
```

4.

```
Select: 4
----- TYPE 4 -----
** Which vendor supplies the most products across the chain, and how many total units have been sold
? **

--- Query Result ---
vendor_id      name              num_products      total_units
5             FreshMart Supply  3                  11
```

5.

```
Select: 5
----- TYPE 5 -----
** Which products in each store are below the reorder threshold and need restocking? **

--- Query Result ---
store_id      store_name      upc              product_name      stock_amount      reorder_threshold
1             Downtown Branch 1001234567893    sprite            8                 10
1             Downtown Branch 1001234567895    triangle kimbap   12                15
1             Downtown Branch 1001234567899    ice cream         3                 5
2             University Store 1001234567890    black coffee      18                20
3             City Center      1001234567890    black coffee      5                 20
3             City Center      1001234567891    milk coffee       7                 20
7             Hospital Branch  1001234567890    black coffee      15                20
7             Hospital Branch  1001234567891    milk coffee       18                20
```

6.

```
Select: 6
----- TYPE 6 -----
** List the top 3 items that loyalty program customers typically purchase with coffee. **
Enter a product name (black coffee or milk coffee): black coffee

--- Query Result ---
product_name      co_purchase_count
milk coffee       8
cola              4
triangle kimbap   3
```

```

Select: 6
----- TYPE 6 -----
** List the top 3 items that loyalty program customers typically purchase with coffee. **
Enter a product name (black coffee or milk coffee): milk coffee

--- Query Result ---
product_name      co_purchase_count
black coffee      8
potato chips      3
ice cream         2

```

7.

```

Select: 7
----- TYPE 7 -----
** Among franchise-owned stores, which one offers the widest variety of products, and how does that compare to corporate-owned stores? **

--- Query Result ---
f_store_id      f_store_name      f_product_variety      c_store_id      c_store_name      c_product_variety
2               University Store    10                     5               Shopping Mall      15

```

0.

```

Select: 0
Program terminated.

```

모든 테스트 케이스에 대해 올바른 결과를 출력하는 것을 볼 수 있습니다.

• Validation of business rule enforcement

위에서 언급한 바와 같이 가격 양수 제약, 재고 음수 방지 등 데이터 무결성이 잘 지켜졌습니다. 또, 참조 무결성 규칙과 도메인 제약 규칙도 잘 지켜지고 있습니다.

비즈니스 로직에 대해 더 자세히 검증해보자면, 임계값 이하 재고를 정확히 식별하는 재주문 로직이 잘 작동하고, 기간별/매장별 집계가 정확하게 이루어지고 있습니다. 또한, 고객 등급 별 분석이 가능하므로 충성도 기반 구매 패턴을 분석하는 실제 비즈니스 로직을 반영하고 있습니다.